

LBaaS Positioning & Demand Hypothesis

A concise product framing for self-service load balancing: the customer job, value proposition, measurable success, and experiment plan.

Target user: Application Owner / Service Owner



Positioning statement

Who it is for, what it replaces, and why it is different.

For application owners and platform teams who need to manage application traffic across enterprise environments, LBaaS is a self-service load balancing platform that enables governed creation, modification, and lifecycle management of LTM and GTM configurations.

Unlike the traditional ticket-based network operations process, LBaaS allows authorized users to complete common load balancing workflows through guided UI and API automation, reducing delays, rework, and operational dependency while maintaining validation, access control, and reliability guardrails.

Target

**Application owners +
platform teams**

Category

**Self-service load
balancing platform**

Alternative

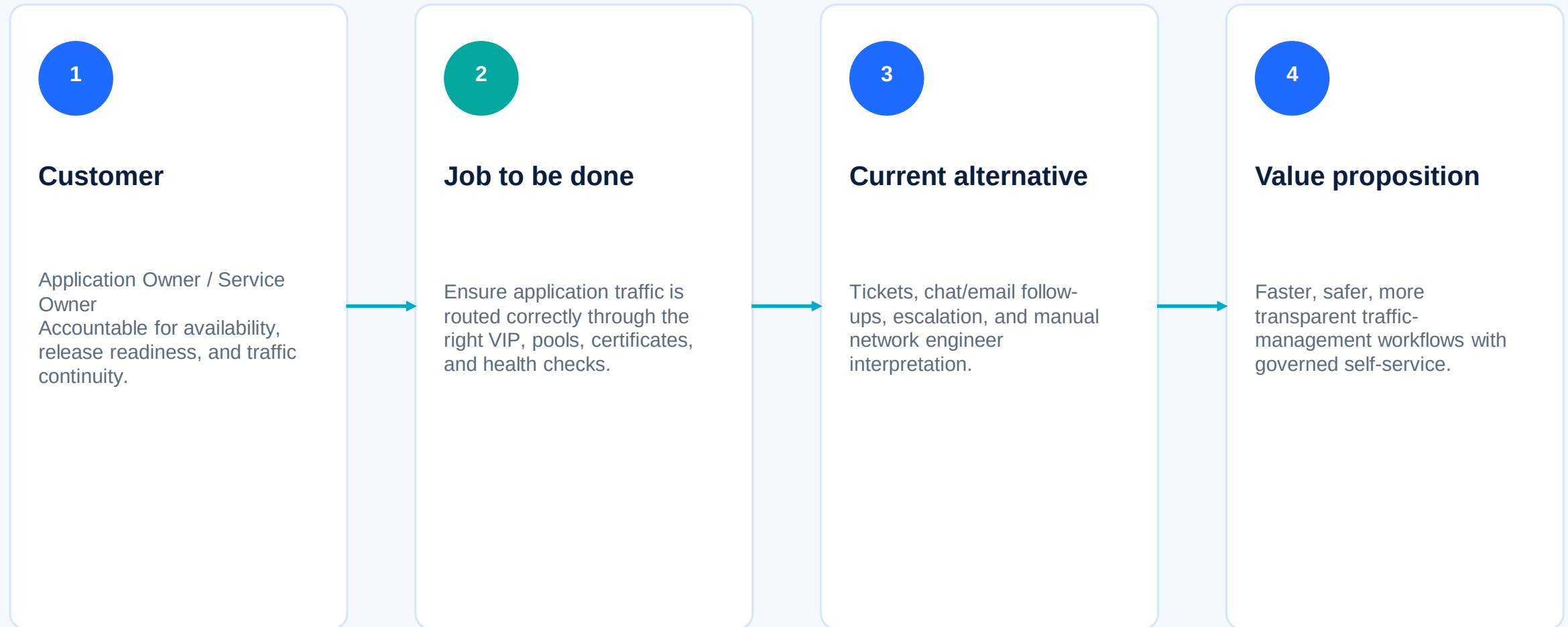
Ticket-based network ops

Difference

**Guided automation +
guardrails**

Demand / value hypothesis map

The job is the desired outcome; LBaaS is the proposed way to get there.



Focal questions

What we need to learn before scaling the product experience.



Problem / JTBD

Can application owners complete the traffic-routing job without deep network knowledge?



Value

Does guided self-service reduce waiting, rework, and status chasing?



Trust

Do users trust the workflow for release-readiness decisions?



Boundaries

Which changes still require network engineer approval or review?



Adoption

Will users return to LBaaS for repeat VIP, pool, certificate, or health-monitor changes?

Value proposition in testable form

The hypothesis must describe observable customer behavior.

If we provide authorized application owners with a guided LBaaS workflow for creating and modifying load balancing configurations, then they will complete common VIP and pool-management tasks faster, with fewer support tickets and fewer configuration errors than the current ticket-based process.



Complete workflow without opening a manual ops ticket



Correct validation issues without support



Use LBaaS again for repeat traffic changes



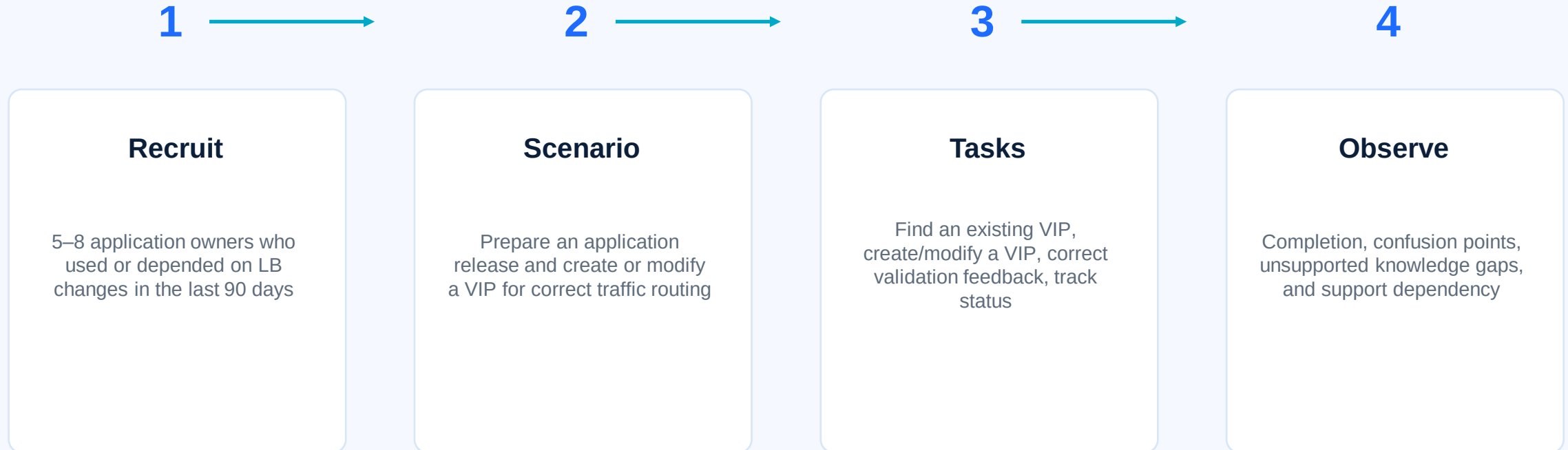
Track status without chat/email follow-ups



Validate release readiness before production

Experiment sketch: guided workflow pilot

Use a prototype or non-production pilot to test whether the job gets done.



Experiment goal: prove users can complete a common load-balancing job with less time, less rework, and less manual operational dependency.

How success is measured

Metrics should indicate adoption, speed, quality, and confidence.

80%+

task completion rate

30%+

faster than comparable ticket flow

70%+

users correct validation errors
unaided

4/5+

average confidence rating



support dependency and
clarification loops



missing-field and rework rates

Team logistics

A lightweight operating model for running the pilot and scaling learnings.

Product

Own hypothesis, participant selection, acceptance criteria, and success readout

Engineering

Prototype / pilot implementation, validation rules, API/UI feasibility, telemetry hooks

Network / SRE

Guardrails, approval boundaries, operational review, failure handling, rollback model

Users / Support

Pilot feedback, task observations, recurring pain points, adoption blockers

Cadence: weekly pilot review • Artifact: experiment readout • Decision: iterate, expand pilot, or revise workflow assumptions